



МИНОБРНАУКИ РОССИИ

**федеральное государственное автономное образовательное учреждение
высшего образования**

«Московский государственный технологический университет

«СТАНКИН»

(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных
технологий

Кафедра
информационных систем

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

Отчет по дисциплине «Структуры и алгоритмы обработки данных»

Тема: «Лабораторная работа №1»

**Проверил
К.т.н., доцент**

Ковалев И.А.

подпись

**Выполнил
студент группы ИДБ-25-06**

Грачев А.А.

подпись

Москва, 2026 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. КЛАСС ИЗМЕРЕНИЯ ВРЕМЕНИ.....	4
ГЛАВА 2. ИЗМЕРЕНИЕ ФУНКЦИЙ	5
2.1. ОБРАЩЕНИЕ К СЕРЕДИННОМУ ЭЛЕМЕНТУ ПО ИНДЕКСУ	5
2.2. ВЫВОД МАКСИМАЛЬНОГО ЭЛЕМЕНТА В МАССИВЕ	5
2.3. БЫСТРАЯ СОРТИРОВКА	6
2.4. СОРТИРОВКА ПУЗЫРЬКОМ	7
2.5. НАИВНОЕ ВЫЧИСЛЕНИЕ ЧИСЛА ФИБОНАЧЧИ	7
2.6. БИНАРНЫЙ ПОИСК.....	8
2.7. ПОЛНЫЙ КОД ПРОГРАММЫ	9
ЗАКЛЮЧЕНИЕ.....	14

ВВЕДЕНИЕ

Целью работы является изучение сложности алгоритмов, а также работа временем их выполнения.

ГЛАВА 1. КЛАСС ИЗМЕРЕНИЯ ВРЕМЕНИ

```
class Timer {
    std::chrono::high_resolution_clock::time_point start;
    std::string name;

public:
    Timer(const std::string& n = "") : name(n) {
        start = std::chrono::high_resolution_clock::now();
    }

    ~Timer() {
        auto end = std::chrono::high_resolution_clock::now();
        auto duration =
std::chrono::duration_cast<std::chrono::microseconds>(end - start);
        if (name != "")
            std::cout << name << ": " << duration.count() << " μs\n";
        else
            std::cout << duration.count() << std::endl;
    }
};
```

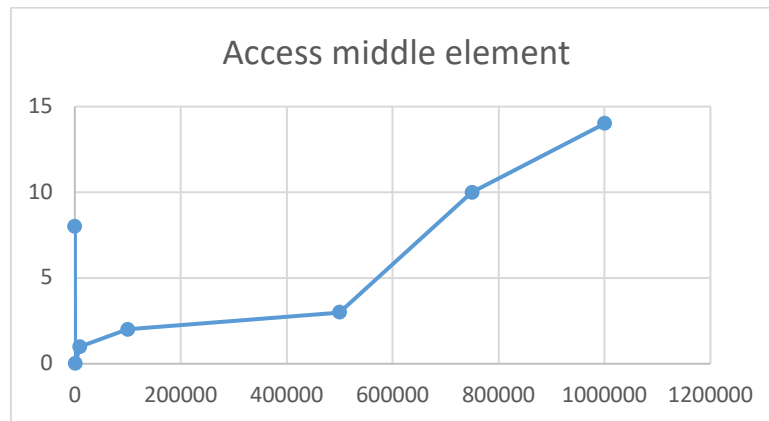
С помощью данного класса после деструктора выводится время работы в микросекундах.

Внутри примененного класса можно поместить функцию и измерить время ее выполнения. Так же через этот класс прошли несколько разных по размеру функций в разных примерах и были выведены результаты времени работы.

ГЛАВА 2. ИЗМЕРЕНИЕ ФУНКЦИЙ

2.1. ОБРАЩЕНИЕ К СЕРЕДИННОМУ ЭЛЕМЕНТУ ПО ИНДЕКСУ

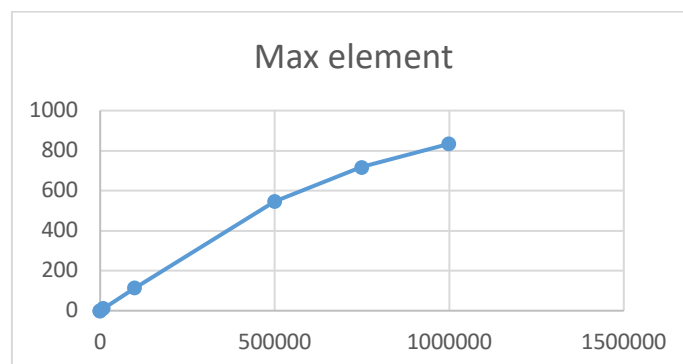
```
// O(1)
int access_middle( int *a, int n)
{
    return a[n / 2];
}
```



Эта функция выводит срединный элемент, сложность функции – $O(1)$.

2.2. ВЫВОД МАКСИМАЛЬНОГО ЭЛЕМЕНТА В МАССИВЕ

```
int find_max( int *a, int n)
{
    int mx = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > mx) mx = a[i];
    return mx;
}
```



Эта функция выводит максимальный элемент;
сложность функции – $O(n)$.

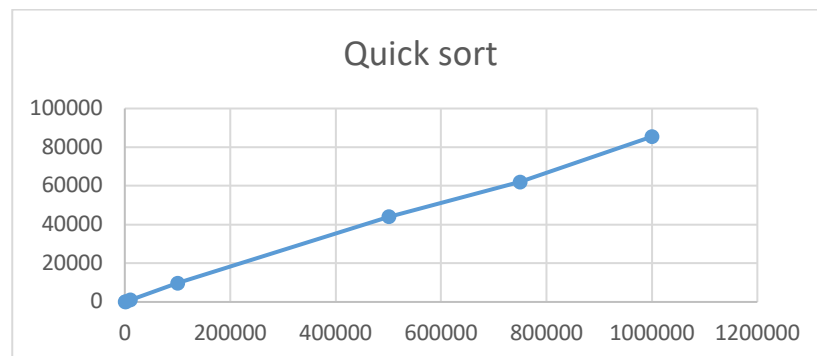
2.3. БЫСТРАЯ СОРТИРОВКА

```

/* --- вспомогательные для quick sort (схема Хоара) --- */
static void qs_rec(int *a, int lo, int hi)
{
    if (lo >= hi) return;
    int pivot = a[lo + (hi - lo) / 2]; // опорный – средний элемент
    int i = lo, j = hi;
    while (i <= j) {
        while (a[i] < pivot) i++;
        while (a[j] > pivot) j--;
        if (i <= j) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++; j--;
        }
    }
    qs_rec(a, lo, j);
    qs_rec(a, i, hi);
}

/* 15. Быстрая сортировка (quick sort)
 *
 * Выбираем опорный элемент, разделяем массив на две части
 * (< pivot и > pivot), рекурсивно сортируем каждую.
 * В среднем  $O(n \log n)$ , в худшем  $O(n^2)$ .
 */
//  $O(n \log n)$  в среднем,  $O(n^2)$  в худшем случае
void quick_sort(int *a, int n)
{
    qs_rec(a, 0, n - 1);
}

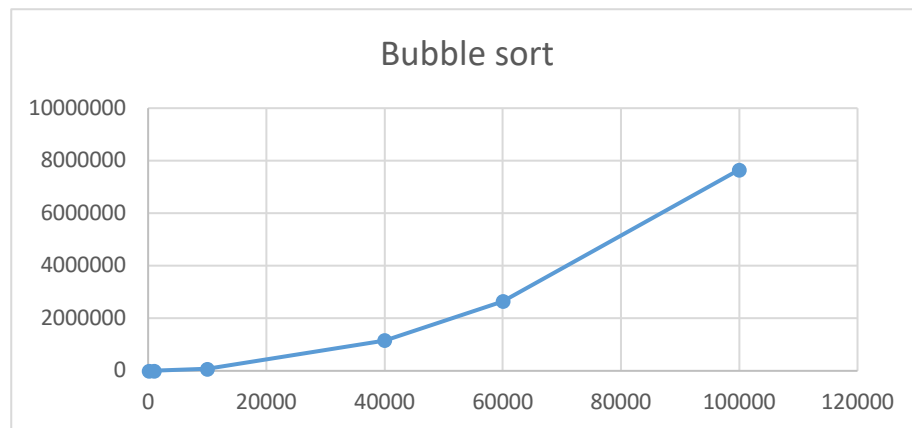
```



Эта функция сортирует массив;
 сложность функции – $O(n \cdot \log n)$.

2.4. СОРТИРОВКА ПУЗЫРЬКОМ

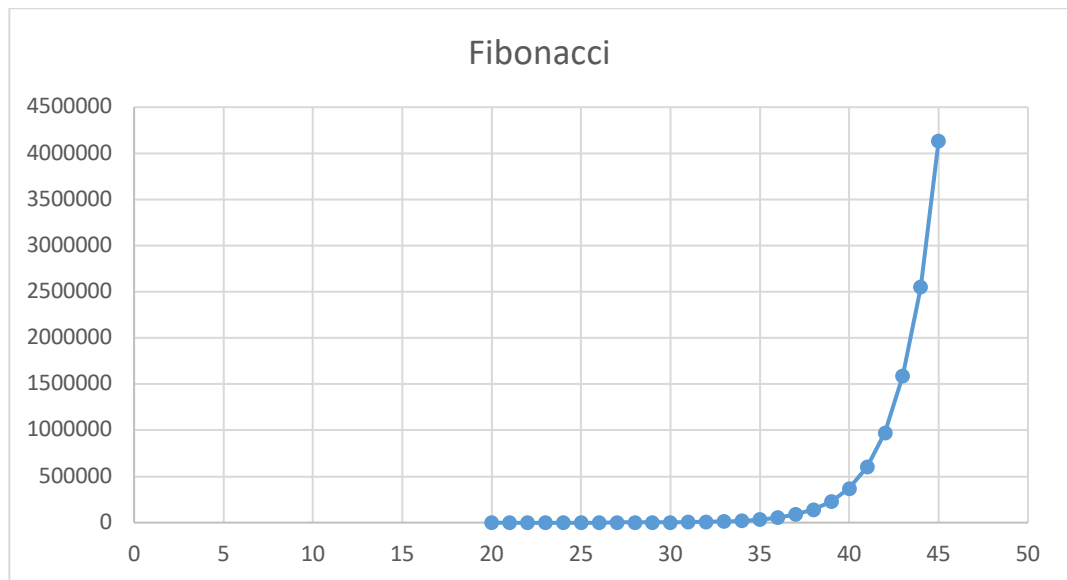
```
void bubble_sort(int *a, int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
}
```



Эта функция выводит сортирует массив;
 сложность функции – $O(n^2)$.

2.5. НАИВНОЕ ВЫЧИСЛЕНИЕ ЧИСЛА ФИБОНАЧЧИ

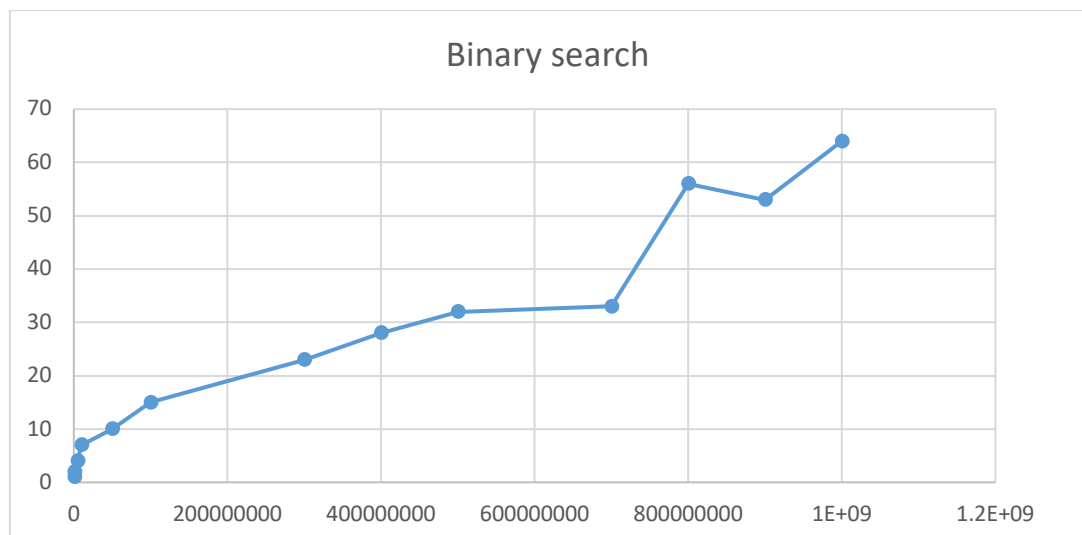
```
long long fib_naive(int n)
{
    if (n <= 1) return n;
    return fib_naive(n - 1) + fib_naive(n - 2);
}
```



Эта функция выводит число Фибоначчи из исходного числа;
сложность функции – $O(2^n)$.

2.6. БИНАРНЫЙ ПОИСК

```
int binary_search( long long *a, int n, int key)
{
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int m = lo + (hi - lo) / 2;
        if (a[m] == key) return m;
        else if (a[m] < key) lo = m + 1;
        else hi = m - 1;
    }
    return -1;
}
```



2.7. ПОЛНЫЙ КОД ПРОГРАММЫ

```

#include <chrono>
#include <iostream>
#include <ctime>
#include <string>
/*
g++ lab1.cpp -o lab1
./lab1
*/
class Timer {
    std::chrono::high_resolution_clock::time_point start;
    std::string name;

public:
    Timer(const std::string& n = "") : name(n) {
        start = std::chrono::high_resolution_clock::now();
    }

    ~Timer() {
        auto end = std::chrono::high_resolution_clock::now();
        auto duration =
std::chrono::duration_cast<std::chrono::microseconds>(end - start);
        if (name != "")
            std::cout << name << ": " << duration.count() << " μs\n";
        else
            std::cout << duration.count() << std::endl;
    }
};

// Массив из случайных чисел
int* randArr(int col, int min, int max) {
    if (min > max) {
        int t = min;
        min = max;
        max = t;
    }
    srand(time(NULL));
    int* newArr = new int[col];
    for (int i = 0; i < col; i++) {
        newArr[i] = min + rand()%(max - min + 1);
    }
    return newArr;
}

long long* randArr2(int size, int minVal, int maxVal) {
    long long int* arr = new long long int[size];
    for (int i = 0; i < size; i++) {
        arr[i] = minVal + rand() % (maxVal - minVal + 1);
    }
}

```

```

    }
    return arr;
}
long long* generateSortedArray(int size, int start, int maxStep) {
    if (size <= 0) return nullptr;

    long long* arr = new long long[size];

    // инициализация генератора (вызвать 1 раз!)
    srand(time(NULL));

    arr[0] = start;

    for (int i = 1; i < size; i++) {
        int step = rand() % maxStep + 1; // шаг от 1 до maxStep
        arr[i] = arr[i - 1] + step;
    }

    return arr;
}
long long getRandomElement(long long* arr, int size) {
    if (size <= 0) {
        throw std::runtime_error("Array is empty");
    }

    int index = rand() % size;
    return arr[index];
}
// O(1)
int access_middle( int *a, int n)
{
    return a[n / 2];
}

// O(log n)
int binary_search( long long *a, int n, int key)
{
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int m = lo + (hi - lo) / 2;
        if (a[m] == key) return m;
        else if (a[m] < key) lo = m + 1;
        else hi = m - 1;
    }
    return -1;
}
// O(n)
int find_max( int *a, int n)

```

```

{
    int mx = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > mx) mx = a[i];
    return mx;
}
/* --- вспомогательные для quick sort (схема Хоара) --- */
static void qs_rec(int *a, int lo, int hi)
{
    if (lo >= hi) return;
    int pivot = a[lo + (hi - lo) / 2]; // опорный — средний элемент
    int i = lo, j = hi;
    while (i <= j) {
        while (a[i] < pivot) i++;
        while (a[j] > pivot) j--;
        if (i <= j) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++; j--;
        }
    }
    qs_rec(a, lo, j);
    qs_rec(a, i, hi);
}
/* 15. Быстрая сортировка (quick sort)
 *
 * Выбираем опорный элемент, разделяем массив на две части
 * (< pivot и > pivot), рекурсивно сортируем каждую.
 * В среднем  $O(n \log n)$ , в худшем  $O(n^2)$ .
 */
//  $O(n \log n)$  в среднем,  $O(n^2)$  в худшем случае
void quick_sort(int *a, int n)
{
    qs_rec(a, 0, n - 1);
}
//  $O(n^2)$ 
void bubble_sort(int *a, int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
}

/* 29. Наивное вычисление числа Фибоначчи —  $O(2^n)$ 
 *
 * Классический пример экспоненциальной рекурсии:
 * fib(n) = fib(n-1) + fib(n-2), без мемоизации.
 */

```

```

*   Дерево рекурсии имеет  $\sim 2^n$  вершин.
*
*   (Аргумент — не массив, а число `n`.)
*/
// 0(2^n)
long long fib_naive(int n)
{
    if (n <= 1) return n;
    return fib_naive(n - 1) + fib_naive(n - 2);
}

int main(){
    int k = 8;
    const int sizes[7] = {100, 1000, 10000, 100000, 500000, 750000, 1000000};
    const int sizes3[6] = {100, 1000, 10000, 40000, 60000, 100000};
    const int sizes2[13] = {750000, 1000000, 5000000, 10000000, 50000000,
100000000, 300000000, 400000000, 500000000, 700000000, 800000000, 900000000,
1000000000};
    std::cout<<"Access middle element:\n";
    std::cout<<"size time(mcrs)\n";
    for (int size : sizes){
        int* arr = randArr(size, 1, 1000000);
        {
            Timer t{};
            access_middle(arr, size);
            std::cout << size << "    ";
        }
        delete[] arr;
    }

    std::cout<<"Max element:\n";
    std::cout<<"size time(mcrs)\n";
    for (int size : sizes){
        int* arr = randArr(size, 1, 1000000);
        {
            Timer t{};
            find_max(arr, size);
            std::cout << size << "    ";
        }
        delete[] arr;
    }

    std::cout<<"Quick sort:\n";
    std::cout<<"size time(mcrs)\n";
    for (int size : sizes){
        int* arr = randArr(size, 1, 1000000);
        {
            Timer t{};

```

```

        quick_sort(arr, size);
        std::cout << size << "    ";
    }
    delete[] arr;
}
std::cout<<"Bubble sort:\n";
std::cout<<"size time(mcrs)\n";
for (int size: sizes3){
    int* arr = randArr(size, 1, 1000000);
    {
        Timer t{};
        bubble_sort(arr, size);
        std::cout << size << "    ";
    }
    delete[] arr;
}
std::cout<<"Fibonacci:\n";
std::cout<<"size time(mcrs)\n";
for (int size = 20; size <= 45; size += 1){
    {
        Timer t{};
        fib_naive(size);
        std::cout << size << "    ";
    }
}
std::cout<<"Binary search:\n";
std::cout<<"size time(mcrs)\n";
for (int size : sizes2){
    long long* arr = generateSortedArray(size, 1, 10);
    {
        Timer t{};
        binary_search(arr, size, getRandomElement(arr, size)); // Ищем
        несуществующий элемент, чтобы гарантировать худший случай
        std::cout << size << "    ";
    }
    delete[] arr;
}
return 0;
}

```

ЗАКЛЮЧЕНИЕ

В данной лабораторной работе были изучены различные алгоритмы, их сложности и время выполнения. Было изучено, как сложность алгоритма влияет на скорость работы программы в зависимости от входных данных.